MODULE 08 · DOMAIN 2 - ACCOUNT MANAGEMENT AND DATA GOVERNANCE

The object hierarchy and the RBAC model

Domain 2 begins. The role hierarchy diagram is the highest-value slide in this domain.

DOMAIN 2 of 5 WEIGHT 20% MODULE 8 of 31

admin@sthithapragna.com

Privileges are granted to **roles**. Roles are granted to **users**. Almost nothing is granted to a user directly.

The object hierarchy

01

Organization

One or more accounts, managed together. Billing and account lifecycle.

›

02

Account

One cloud provider, one region. Its own users, roles and warehouses.

›

03

Database

A logical container. Nothing is stored at this level.

›

04

Schema

Contains the objects. Tables, views, stages, functions, procedures.

ON THE EXAM

Every securable database object lives in a schema, inside a database. That containment is what makes USAGE privileges necessary.

Two access control models, working together

- **Discretionary Access Control.** Every object has an **owner**, and the owner can grant access to what it owns.
- **Role-Based Access Control.** Privileges are assigned to roles, and roles are assigned to users or to other roles.
- Snowflake uses **both**. Ownership decides who may grant; roles decide who may act.
- Newer: **user-based access control** lets a privilege be granted to a user. It only takes effect when **all secondary roles are enabled**.

> ON THE EXAM
>
> Disable UBAC if it complicates governance: ALTER ACCOUNT SET DISABLE_USER_PRIVILEGE_GRANTS = TRUE.

The system-defined roles

ROLE	WHAT IT IS FOR
ORGADMIN	Organization-level tasks. Account lifecycle and organization usage. Outside the account hierarchy.
ACCOUNTADMIN	Encapsulates SYSADMIN and SECURITYADMIN. Top-level. Account parameters, billing, killing queries.
SECURITYADMIN	Global MANAGE GRANTS: can grant or revoke on any object. Inherits USERADMIN.
USERADMIN	CREATE USER and CREATE ROLE. User and role management only.
SYSADMIN	Create warehouses, databases and all database objects.
PUBLIC	Pseudo-role granted automatically to every user and every role.

ON THE EXAM

Read the two middle rows carefully. USERADMIN creates users and roles. SECURITYADMIN grants privileges. Different jobs.

The hierarchy, bottom to top

01

USERADMIN

Granted to SECURITYADMIN.

›

02

SECURITYADMIN

Granted to ACCOUNTADMIN.

›

03

SYSADMIN

Also granted to ACCOUNTADMIN, in parallel.

›

04

ACCOUNTADMIN

Inherits both branches. ORGADMIN sits outside entirely.

ON THE EXAM

Two branches meeting at the top. SECURITYADMIN inherits USERADMIN, so it can do everything USERADMIN can. SYSADMIN does not.

ACCOUNTADMIN is not a superuser

- The documentation says this explicitly. **ACCOUNTADMIN is not a superuser role.**
- It can view and manage an object **only if it, or a role beneath it in a hierarchy, holds sufficient privileges** on that object.
- So an object owned by a custom role that is **not** connected to the hierarchy is invisible to ACCOUNTADMIN too.
- Do not make it anyone's **default role**, and do not use it in **automated scripts**. Both are documented recommendations.
- Snowflake recommends designating **at least two** account administrators.

> ON THE EXAM
>
> Forcing an explicit USE ROLE ACCOUNTADMIN is the point of the default-role advice. It makes the elevation deliberate.

Which role does what: the trap

THE TASK	THE ROLE
Create a user, or create a role	USERADMIN (or higher)
Grant a privilege on an object you do not own	SECURITYADMIN, via MANAGE GRANTS
Create a warehouse, database, schema or table	SYSADMIN (or a custom role with the privilege)
Change an account-level parameter	ACCOUNTADMIN only
View billing and credit usage, or kill a running query	ACCOUNTADMIN only
Create a new account in the organization	ORGADMIN

ON THE EXAM

A question that needs BOTH a user created and a warehouse created needs two roles, or ACCOUNTADMIN. That is a real question shape.

Custom roles, and the SYSADMIN rule

- Build a hierarchy of custom roles aligned to business functions, and grant the **top one to SYSADMIN**.
- If a custom role is **not** assigned to SYSADMIN, **system administrators cannot manage the objects it owns**.
- Objects are owned by **the role active when they were created**. Create under the wrong role and ownership goes to it.
- Ownership can be **transferred** with GRANT OWNERSHIP.

> ON THE EXAM
>
> Creating objects while using ACCOUNTADMIN is the classic accident. The objects end up owned by a role nobody should be using daily.

The commonest access failure

- To query a table you need **SELECT on the table**, plus **USAGE on the schema**, plus **USAGE on the database**.
- You also need **USAGE on a warehouse** to run the query at all.
- Missing container USAGE produces 'I can see it but I cannot query it', or the object appearing not to exist.
- This follows directly from containment. Objects live in schemas, and schemas live in databases.

> ON THE EXAM
>
> Count the levels when a scenario says access was granted and still fails. There are usually three, plus the warehouse.

Future grants

- Define privileges now on objects of a type that **do not exist yet**, in a schema or a database.
- **GRANT SELECT ON ALL TABLES IN SCHEMA covers existing tables only.** ON FUTURE is a separate statement.
- The global **MANAGE GRANTS privilege is required** to grant on future objects in a database.
- **Schema-level future grants take precedence** over database-level ones, and the database-level grants are then ignored.
- **Future grants are not applied when renaming or swapping a table.**

> ON THE EXAM
>
> 'New tables keep being invisible to the analysts' is the scenario. ON ALL was run once; ON FUTURE never was.

Managed access schemas

- Created with **CREATE SCHEMA … WITH MANAGED ACCESS**.
- **Object owners lose the ability to make grant decisions** inside that schema.
- Only the **schema owner**, or a role with **MANAGE GRANTS**, can grant on objects in it, including future grants.
- The point is **centralising privilege management**, so grants cannot proliferate quietly.

> ON THE EXAM
>
> This is the one place Snowflake deliberately switches off the discretionary half of the model.

Primary and secondary roles

- A session has **one primary role**, set with USE ROLE. **It owns anything you create.**
- **Secondary roles** add their privileges to the session without changing ownership.
- **USE SECONDARY ROLES ALL** activates every role the user holds. **NONE** disables them.
- Useful when a query joins objects governed by different roles, which would otherwise need a combined role.

> ON THE EXAM
>
> Secondary roles never own anything. If a scenario asks who owns a new table, it is the primary role at creation time.

Account roles and database roles

- **Account roles** are what we have been discussing. They exist across the account and can be activated in a session.
- **Database roles** are **scoped to a single database** and hold privileges only on objects in it.
- **A database role cannot be activated directly in a session.** Grant it to an account role, then use that.
- That rule explains something from module 6: SNOWFLAKE.CORTEX_USER is a database role, which is why it cannot be granted to a user.

> ON THE EXAM
>
> Database roles travel with the database. That makes them the natural unit for sharing and for native apps.

ORGADMIN is being replaced

- **ORGADMIN** is the role that uses a regular account to manage organization-level operations. It is what the exam will ask about.
- The documentation now states that **ORGADMIN will be phased out in a future release**.
- Its replacement is **GLOBALORGADMIN**, which exists **only in the organization account**, a dedicated account for organization administration.
- **Answer ORGADMIN.** The January 2026 study guide predates the transition.

> ON THE EXAM
>
> Recognise GLOBALORGADMIN so it does not throw you, but do not expect it to be the keyed answer.

A new data engineer is being onboarded. The onboarding runbook requires creating a Snowflake user for them, and a custom role for the engineering team. It also requires a dedicated virtual warehouse for their workload.

An administrator attempts all three while using the SECURITYADMIN role. The first two succeed and the third fails.

Why did the third step fail, and what is the correct approach?

A SECURITYADMIN cannot create warehouses. Use SYSADMIN for that step, or a custom role with the privilege.

B Warehouse creation requires ACCOUNTADMIN, because warehouses consume credits.

C SECURITYADMIN must first be granted to SYSADMIN so it inherits object creation privileges.

D The administrator must enable secondary roles, which would have granted the missing privilege automatically.

◆ PAUSE AND DECIDE

Why did the third step fail, and what is the correct approach?

A SECURITYADMIN cannot create warehouses. Use SYSADMIN.

SECURITYADMIN inherits USERADMIN, which is why the first two steps worked. It has no object-creation branch at all.

B Warehouse creation needs ACCOUNTADMIN.

SYSADMIN creates warehouses. ACCOUNTADMIN would work only because it inherits SYSADMIN, which makes it the wrong answer to a 'correct approach' question.

C Grant SECURITYADMIN to SYSADMIN.

Inverts the hierarchy. Granting a role to another gives the grantee the privileges, so this would give SYSADMIN security powers instead.

D Enable secondary roles.

Secondary roles activate roles the user already holds. If they were never granted SYSADMIN, nothing appears.

THE SIGNAL

The first two steps succeeding is the clue. SECURITYADMIN inherits USERADMIN, so user and role creation work. Warehouses are the other branch.

An analyst role has been granted SELECT on a table in SALES.REPORTING. The analyst has USAGE on a warehouse and can log in normally.

When they run a query against the table, Snowflake reports that the object does not exist or is not authorised. The table certainly exists, and the grant is visible in SHOW GRANTS.

What is the most likely cause?

A The table is in a managed access schema, which blocks all SELECT grants made by object owners.

B The role is missing USAGE on the SALES database, the REPORTING schema, or both.

C The grant was made with a future grant, which does not apply to tables that already exist.

D The analyst needs their secondary roles enabled before any SELECT grant takes effect.

◆ PAUSE AND DECIDE

What is the most likely cause?

A. Managed access schema blocks the grant.

Managed access restricts who may grant, not whether a granted privilege works. And the grant is visible in SHOW GRANTS.

B. Missing USAGE on the database, the schema, or both.

Objects are contained in schemas, inside databases. USAGE on both containers is required in addition to SELECT.

C. It was a future grant.

Genuinely true that future grants do not apply to existing tables. But then the grant would not appear in SHOW GRANTS for this table.

D. Secondary roles must be enabled.

That applies to privileges granted directly to a user under UBAC. This was granted to a role.

THE SIGNAL

The misleading error message is the point. Missing container USAGE often reports as 'does not exist' rather than 'access denied'.

A nightly pipeline creates several new tables in ANALYTICS.CURATED each week. A platform engineer ran GRANT SELECT ON ALL TABLES IN SCHEMA ANALYTICS.CURATED TO ROLE reporting three months ago.

Reporting users can query the older tables but not any table created since. The engineer is using a custom role that owns the schema but does not hold MANAGE GRANTS.

What should be done?

A Re-run the ON ALL grant after each pipeline run, since Snowflake has no mechanism for future objects.

B Convert the schema to a managed access schema, which automatically grants new objects to existing grantees.

C Add a future grant on the schema. The schema owner can do this without MANAGE GRANTS.

D Grant OWNERSHIP of the pipeline to the reporting role so it inherits access to what it creates.

◆ PAUSE AND DECIDE

What should be done?

A Re-run ON ALL after every run.

Correctly diagnoses that ON ALL covers existing tables only, then denies that future grants exist. They do.

B Convert to a managed access schema.

Managed access centralises who may grant. It does not create grants, so new tables would still be inaccessible.

C Add a future grant on the schema.

Exactly what future grants are for. MANAGE GRANTS is required for future grants on a database; the schema owner can do it at schema level.

D Grant OWNERSHIP of the pipeline.

Ownership of a task or procedure does not confer access to tables it creates. It would also make reporting the owner of production objects.

THE SIGNAL

MANAGE GRANTS is required for future grants at DATABASE level. The scenario mentions the missing privilege to test whether you know the difference.

> An engineer signed in, left their role as ACCOUNTADMIN, and created a database and several tables for a new project. A custom role hierarchy exists, with a top-level role granted to SYSADMIN.
>
> The data team, working under roles beneath SYSADMIN, now cannot see or manage any of the new objects. A team lead argues that ACCOUNTADMIN is the highest role, so its objects should be visible to everyone below it.

Why is the team lead wrong, and what is the fix?

A. ACCOUNTADMIN objects are hidden by design. The database must be dropped and recreated under SYSADMIN.

B. The team roles need SELECT granted on each new table individually. Ownership is irrelevant to visibility.

C. Privileges flow downward from senior roles. The team simply needs their sessions restarted.

D. Privileges are inherited upward, not downward. Transfer ownership to a role in the hierarchy with GRANT OWNERSHIP.

◆ PAUSE AND DECIDE

Why is the team lead wrong, and what is the fix?

A Hidden by design. Drop and recreate.

Not hidden by design, and dropping a populated database to fix a grant is drastic when ownership can simply be transferred.

B Grant SELECT on each table. Ownership is irrelevant.

Would restore reading, and leaves ACCOUNTADMIN owning production objects. Ownership decides who can grant and alter.

C Privileges flow downward. Restart sessions.

States the team lead's misconception. Inheritance runs upward: a parent role inherits from roles granted to it.

D Inheritance is upward. Transfer ownership.

ACCOUNTADMIN is not a superuser, and it inherits from roles granted to it rather than pushing privileges down.

THE SIGNAL

Objects are owned by the role active at creation. This is why 'do not make ACCOUNTADMIN a default role' is documented advice.

What to carry into the next module

- **USERADMIN to SECURITYADMIN to ACCOUNTADMIN**, with **SYSADMIN** as a parallel branch. ORGADMIN sits outside.
- **ACCOUNTADMIN is not a superuser.** Inheritance runs upward, and objects are owned by the role active at creation.
- **SELECT plus USAGE on the schema plus USAGE on the database**, plus a warehouse. Count the levels.
- **ON ALL is existing objects. ON FUTURE is new ones.** Schema-level future grants beat database-level.
- **Managed access:** owners stop granting. **Database roles** cannot be activated directly.

> ON THE EXAM
>
> Next: authentication, network policies and encryption. The area where this course is most obviously newer than its competitors.